



VISUAL STUDY NOTES

System Design for DevOps

Build systems that scale, survive failure and ship safely.
Beginner-friendly. Crisp. Illustrated.

CREATED BY



Sharad Verma

@sharadverma638

Scalability

Reliability

CI/CD

Kubernetes

Observability

IaC

15

sections

25+

illustrations

1

cheat sheet



Want to learn DevOps?



github.com/sharadverma638/sv-opslibrary

Contents

15 short sections. Read in order, or jump to what you need.

01	What is System Design? The big picture and why DevOps cares	2
02	Core Goals Scalability, reliability, availability and friends	3
03	Scaling Scale up, scale out and auto-scaling	4
04	Load Balancing One front door, many servers	5
05	Caching & CDN Answer faster by remembering	6
06	Databases & Data Replication, sharding and the CAP theorem	7
07	Architecture Styles Monolith, microservices and queues	8
08	Containers & Kubernetes Package once, run anywhere	9
09	CI/CD & Deployments Ship small, ship safe	11
10	IaC, GitOps & Environments Infrastructure as reviewed code	12
11	Observability & SRE Logs, metrics, traces, SLOs	13
12	Resilience & Recovery Design for failure, plan for disaster	14
13	Security & Cost Basics Defense in depth, spend wisely	16
14	Putting It All Together A reference architecture and a method	17
15	Cheat Sheet & Roadmap Revise fast, learn next	18

01

What is System Design?

The big picture and why DevOps cares

System design is planning how the parts of a software system (code, servers, databases, networks, pipelines) fit together so it works for real users: **fast, reliable, secure and affordable**.

THINK OF IT LIKE

A city. Roads = network, power plants = servers, warehouses = databases, traffic police = load balancers, CCTV = monitoring. Good design decides where everything goes *before* the city grows.

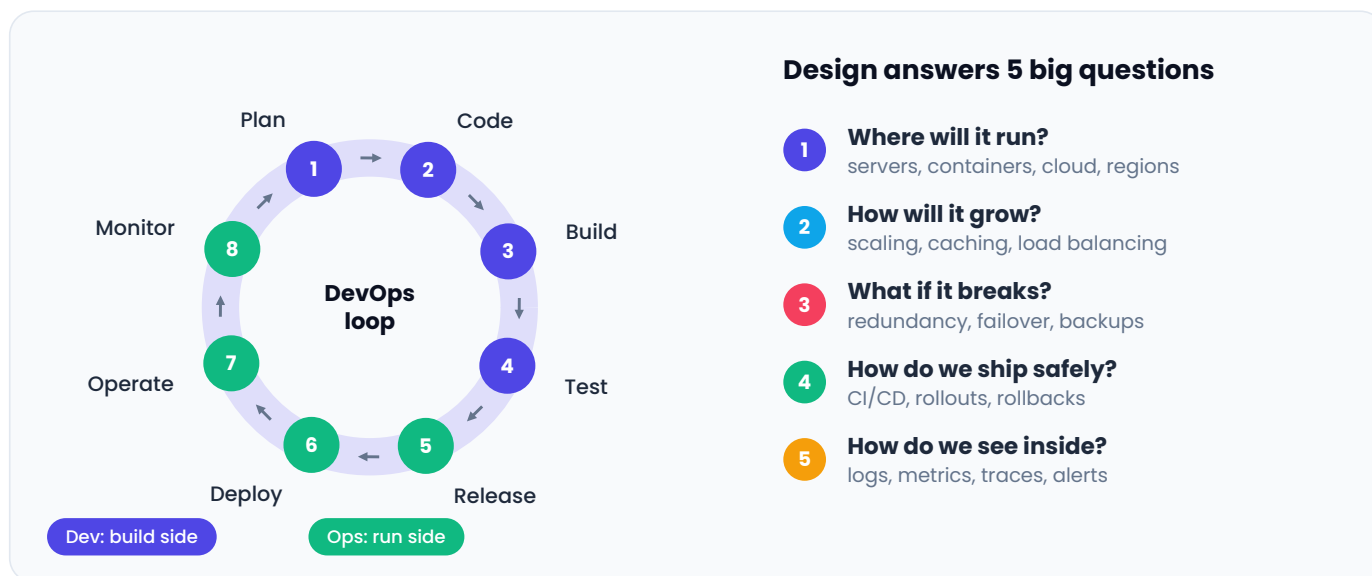


Fig 1. The DevOps loop. System design decides what runs inside every step of it.

Why should a DevOps engineer care?

- **You run it.** When the app slows down at 3 AM, your design choices decide how bad it gets.
- **You pick the building blocks.** Load balancer, cache, queue, database, region: these are design decisions.
- **You make change safe.** Pipelines, rollbacks and monitoring are design too.
- **You control cost.** Over-building wastes money; under-building loses users.

Role	Focus	Question they ask
Developer	Features and code	"Does it work?"
Operations	Servers and uptime	"Does it stay up?"
DevOps	Automation + shared ownership	"Can we ship it safely, often, and keep it healthy?"

02

Core Goals

Scalability, reliability, availability and friends



Before drawing boxes, decide what "good" means. These six goals are the scorecard for every design.

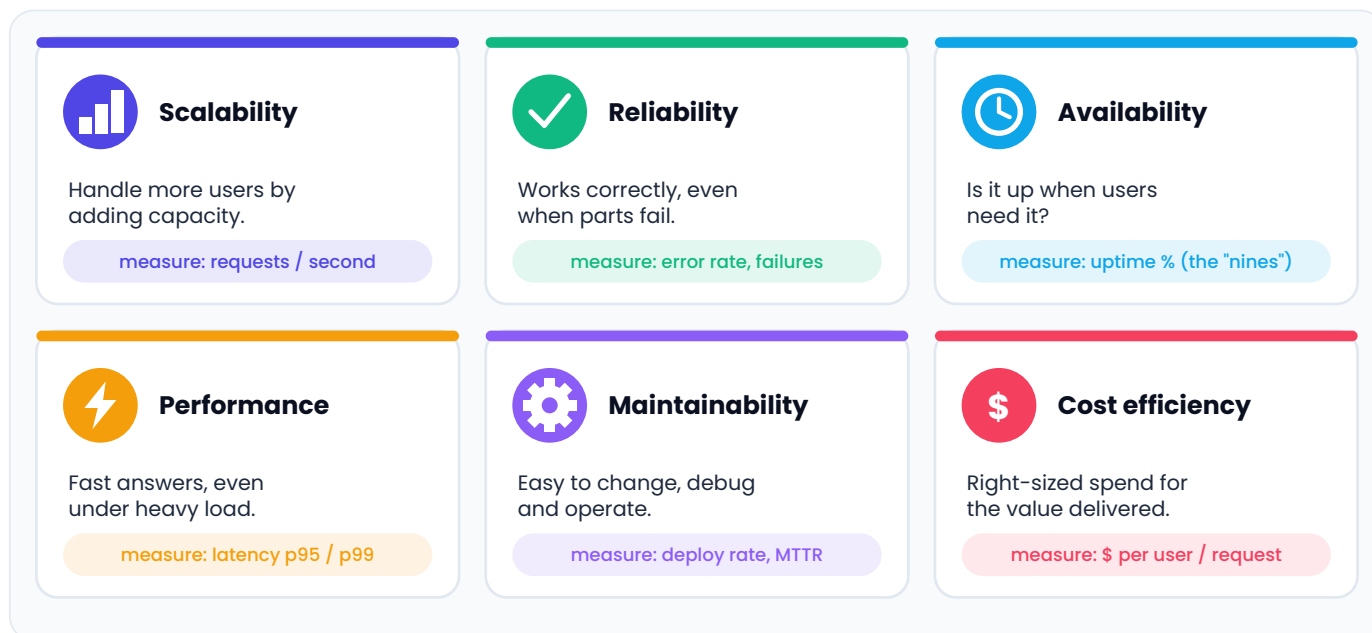


Fig 2. The six goals every design is judged on.

Availability and the "nines"

Availability is the share of time your system works. Each extra "9" is dramatically harder and costlier.

Uptime	Downtime per year	Downtime per month	Typical for
99% (two 9s)	3.65 days	~7.3 hours	Internal tools
99.9% (three 9s)	8.76 hours	~43 minutes	Most web apps
99.99% (four 9s)	52.6 minutes	~4.3 minutes	Payments, core APIs
99.999% (five 9s)	5.26 minutes	~26 seconds	Telecom, critical infra

KEY IDEA

Functional requirements = what it does ("users can pay"). **Non-functional** requirements = how well it does it ("in under 300 ms, 99.9% of the time"). System design is mostly about the second kind.

WATCH OUT

You cannot maximise everything. Faster usually costs more. More consistent can mean less available. Design is the art of **choosing trade-offs on purpose**.

03

Scaling

Scale up, scale out and auto-scaling

Scaling means handling more load. There are two ways to do it.

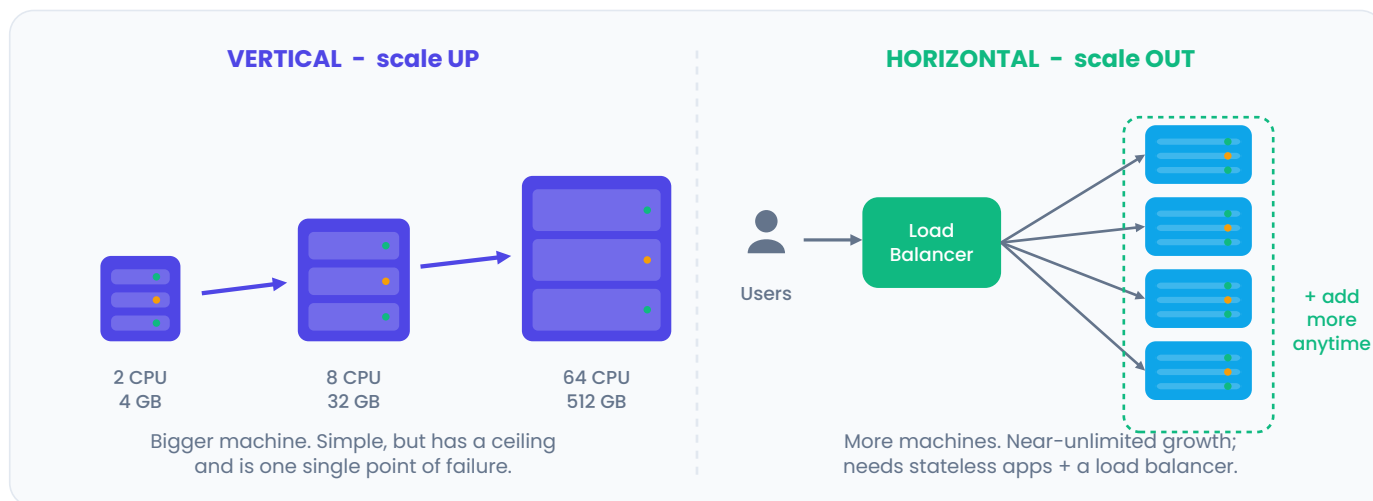


Fig 3. Scale up (bigger machine) vs scale out (more machines).

	Vertical (up)	Horizontal (out)
How	Add CPU / RAM to one server	Add more servers
Pros	Simple, no code changes	No hard ceiling, survives a server dying
Cons	Hardware limit, downtime, single point of failure	Needs load balancer and stateless design
Use when	Quick fix, databases, small apps	Web / API tiers, anything growing

Stateless apps: the key to scaling out

A server is **stateless** when it keeps nothing important in its own memory. Sessions and files live in a shared store (Redis, database, object storage), so **any server can handle any request** and you can add or kill servers freely.

Auto-scaling

Cloud platforms add and remove servers automatically from a rule, for example CPU > 70% for 5 min = add 2 servers. You pay only for what you need.

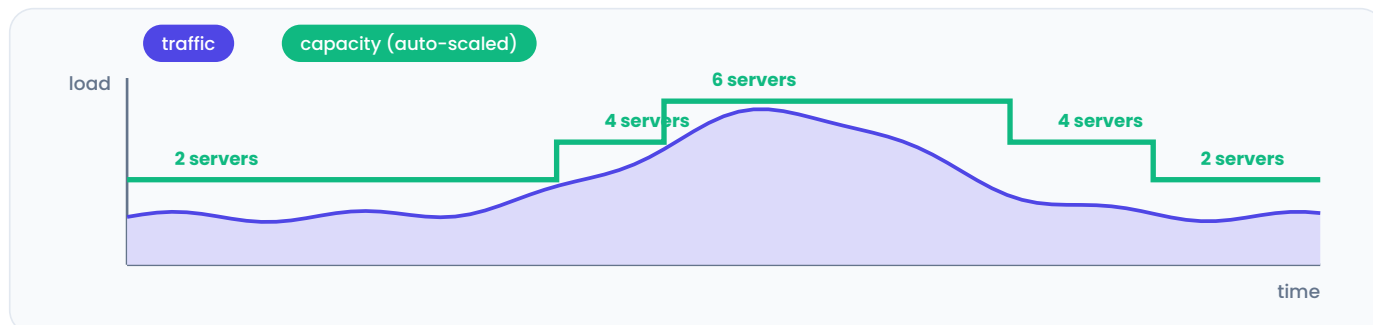


Fig 4. Capacity follows traffic. Scale in again when the spike ends.

04

Load Balancing

One front door, many servers

A **load balancer (LB)** sits in front of your servers, spreads requests across them, and stops sending traffic to any server that fails its health check. It is the front door of a scalable system.

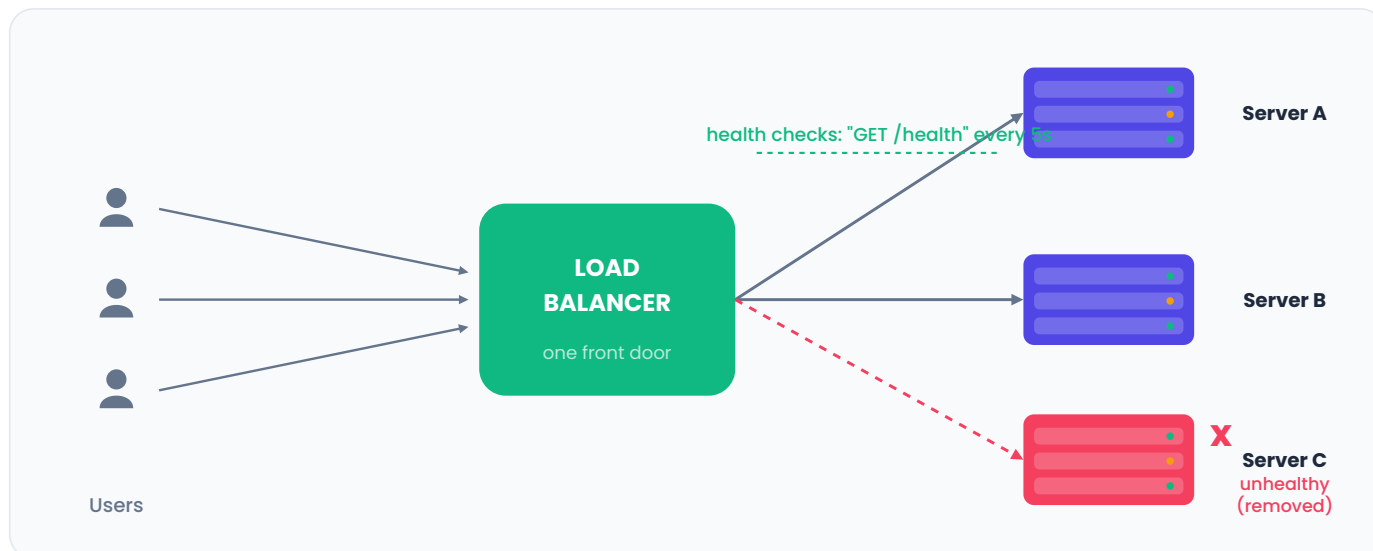


Fig 5. Users talk to one address. The LB shares the work and removes sick servers.

How does it choose a server?

Algorithm	How it works	Best for
Round robin	A, B, C, A, B, C ... in turn	Similar servers, short requests
Weighted round robin	Bigger servers get more turns	Mixed server sizes
Least connections	Pick the server with fewest active requests	Long or uneven requests
IP hash	Same client IP always hits same server	Sticky sessions (use sparingly)
Least response time	Pick the fastest right now	Latency-sensitive apps

Layer 4 vs Layer 7

	L4 (transport)	L7 (application)
Sees	IP address and port only	URLs, headers, cookies
Speed	Very fast, simple	Slightly slower, much smarter
Can do	Forward TCP/UDP	Route /api vs /images, TLS termination, auth
Examples	AWS NLB, HAProxy (tcp)	AWS ALB, Nginx, Envoy, Traefik

WATCH OUT

The load balancer can itself be a single point of failure. Run it as a **redundant pair** or use a managed cloud LB. A **reverse proxy** (like Nginx) does similar work and also adds caching, TLS and security.

05

Caching & CDN

Answer faster by remembering

A **cache** keeps a copy of popular data in a fast place so you skip slow work. A **CDN** (Content Delivery Network) is a cache spread around the world, close to users.

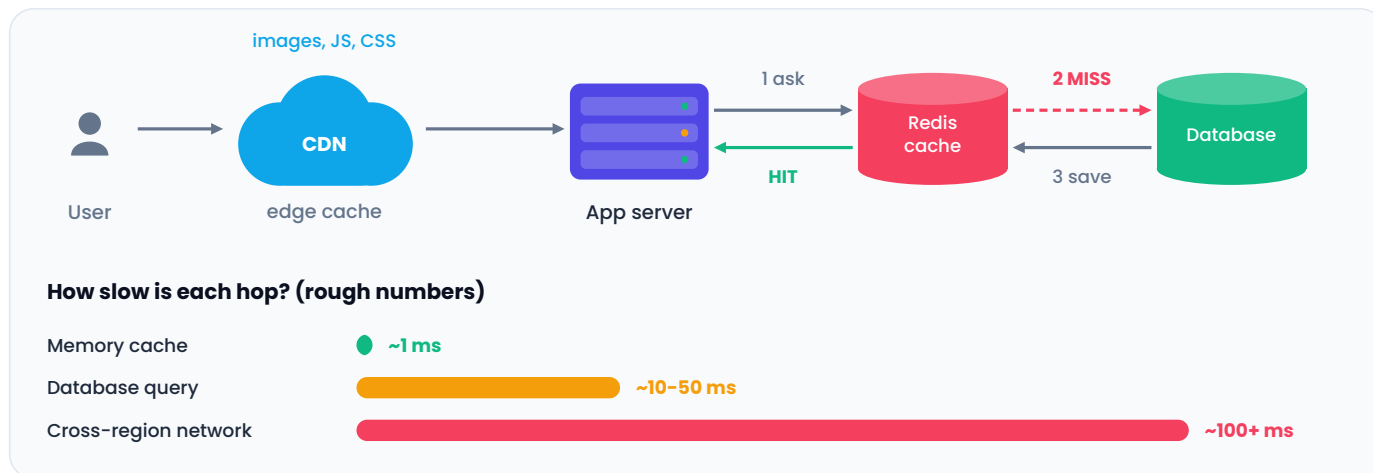


Fig 6. A request meets the CDN, then the app, then Redis, and only then the database.

Common caching strategies

Strategy	What happens	Good to know
Cache-aside	App checks cache; on a miss it reads DB then stores the result	Most popular, simple, may serve stale data
Write-through	Every write goes to cache and DB together	Always fresh, slightly slower writes
Write-back	Write to cache first, DB later in batches	Very fast, risk of loss if cache dies
TTL (expiry)	Each item auto-expires after N seconds	Easiest way to avoid stale data

What to cache, and what to evict

- **Cache:** read-heavy, slow-to-compute, rarely changing data (product pages, profiles, config, query results).
- **Evict** when full: **LRU** removes the Least Recently Used, **LFU** the Least Frequently Used.
- **Layers:** browser cache, CDN, app memory, Redis/Memcached, database cache.
- **Tools:** Redis, Memcached, Cloudflare, CloudFront, Akamai, Varnish.

WATCH OUT

Cache invalidation is the hard part. If data changes, the cache must update or expire, or users see old information. Always set a TTL.

06

Databases & Data

Replication, sharding and the CAP theorem

Data is the hardest thing to scale because it must stay correct. Two big tools: **replication** (copy it) and **sharding** (split it).

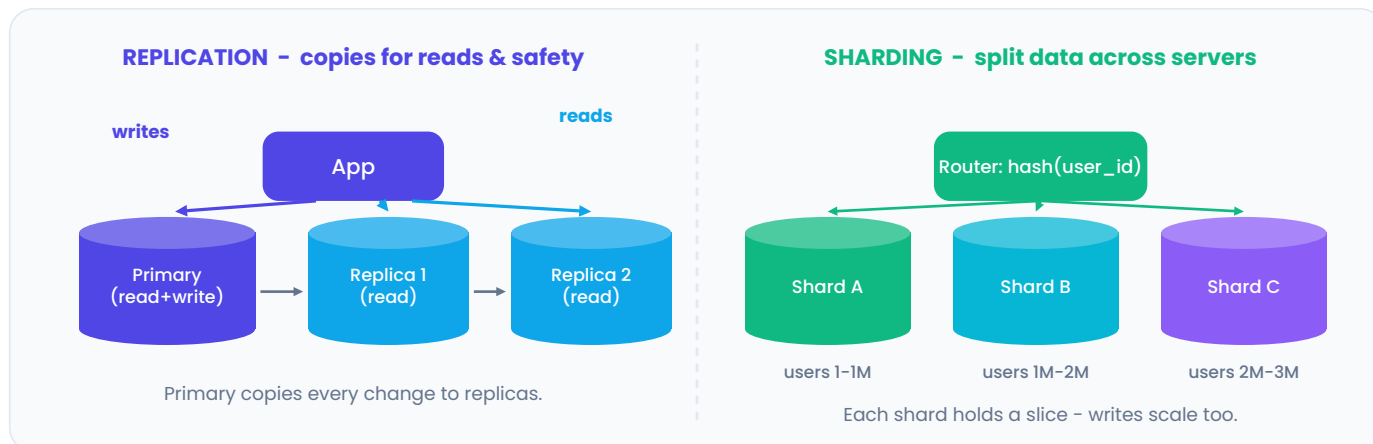


Fig 7. Replication helps reads and safety. Sharding helps writes and size.

	SQL (relational)	NoSQL
Shape	Tables, rows, strict schema	Documents, key-value, wide-column, graph
Strength	Transactions, joins, consistency (ACID)	Flexible schema, easy horizontal scale
Examples	PostgreSQL, MySQL, SQL Server	MongoDB, DynamoDB, Cassandra, Redis
Pick when	Money, orders, anything relational	Huge scale, simple access patterns, flexible data

The CAP theorem



Fig 8. In a distributed system, a network split forces a choice: consistency or availability.

PRO TIP

Replication is not a backup. A bad delete is copied to every replica instantly. Keep separate, versioned, tested backups and practice restoring them.

07

Architecture Styles

Monolith, microservices and queues

How you split your application changes how you build, deploy and operate it.

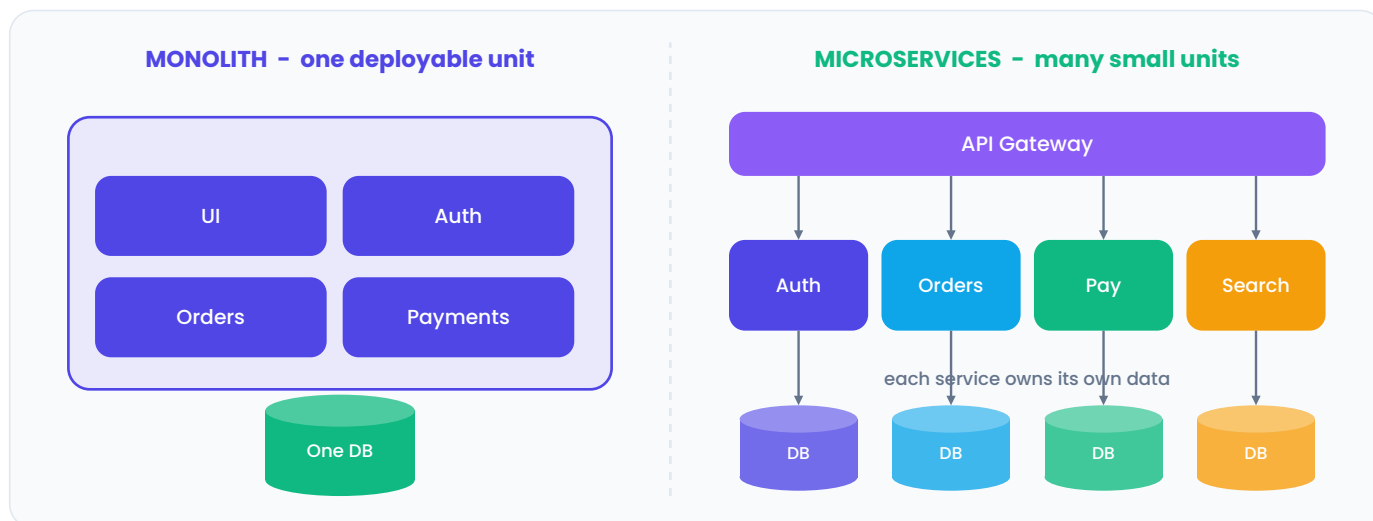


Fig 9. One big unit vs many small services behind an API gateway.

	Monolith	Microservices
Deploy	One unit, all or nothing	Each service on its own
Scale	Scale everything together	Scale only the busy service
Complexity	Low at first, grows with size	High: network, many pipelines, tracing
Best for	Small teams, new products	Large teams, clear domain boundaries

PRO TIP

Start simple. A well-built monolith beats a messy swarm of microservices. Split only when team size or scale demands it.

Async messaging with queues

Instead of waiting for slow work, drop a message in a **queue** and reply instantly. **Workers** pick messages up later. The queue absorbs traffic spikes and keeps working if a worker crashes.

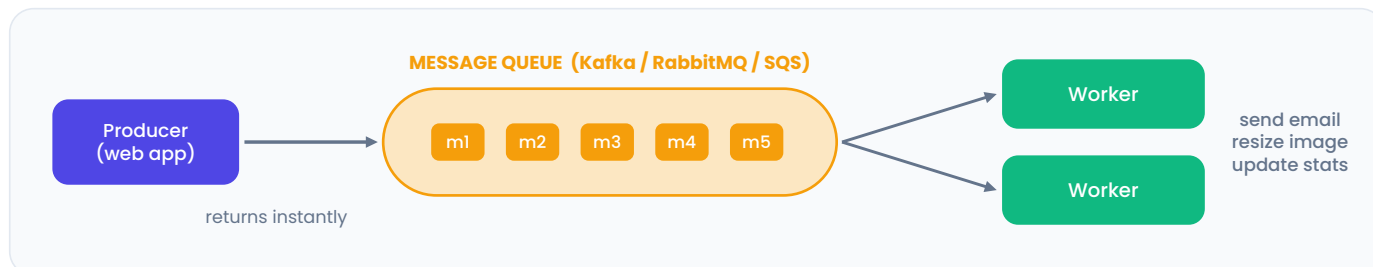


Fig 10. Producer and workers are decoupled by the queue.

Talking style	Examples	Use for
Sync request / response	REST, gRPC, GraphQL	Need the answer right now
Async messages / events	Kafka, RabbitMQ, SQS, Pub/Sub	Background jobs, spikes, event streams

08

Containers & Kubernetes

Package once, run anywhere



A **container** bundles your app with everything it needs, so it runs the same on a laptop, in testing and in production. It shares the host's kernel, so it is far lighter than a virtual machine.

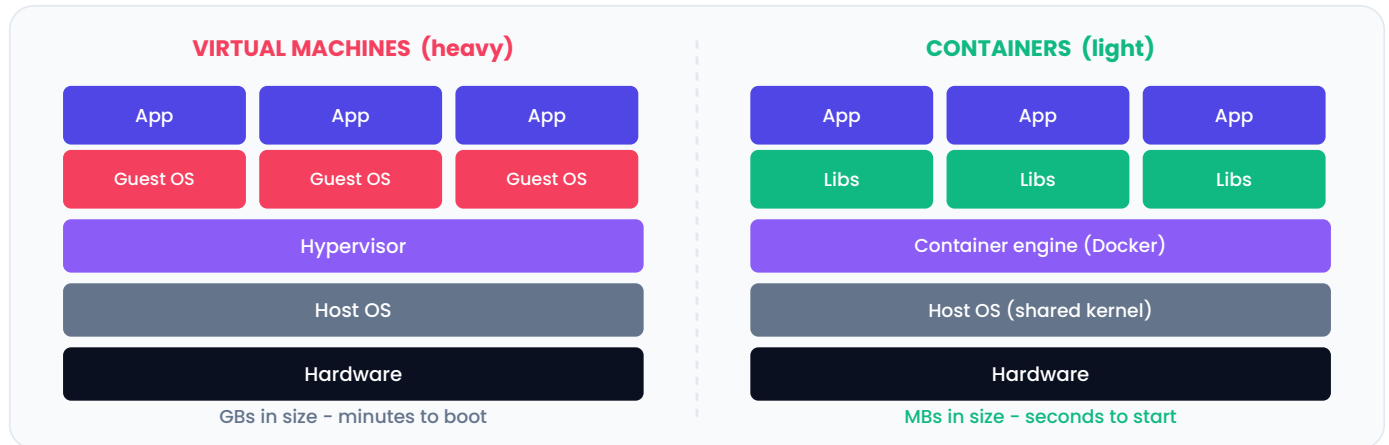


Fig 11. Containers skip the guest OS, so they start in seconds.

A Dockerfile is the recipe

```
# Dockerfile (the recipe for an image)
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node", "server.js"]
```

Term	Meaning
Image	A read-only template built from the Dockerfile
Container	A running instance of an image
Registry	Where images are stored and shared (Docker Hub, ECR, GHCR)

Good image habits

- **Small base images** (alpine, distroless) are faster and have fewer vulnerabilities.
- **One process per container**, and keep containers disposable.
- **Pin versions** (node:20.11) instead of using "latest".
- **Run as non-root**, and keep secrets out of the image.
- **Scan images** in your pipeline before they ship.

Kubernetes (K8s): the container manager

Running a few containers is easy. Running hundreds is not. **Kubernetes** schedules them across machines, restarts failed ones, scales them and rolls out updates. You declare the *desired state*; it makes reality match.

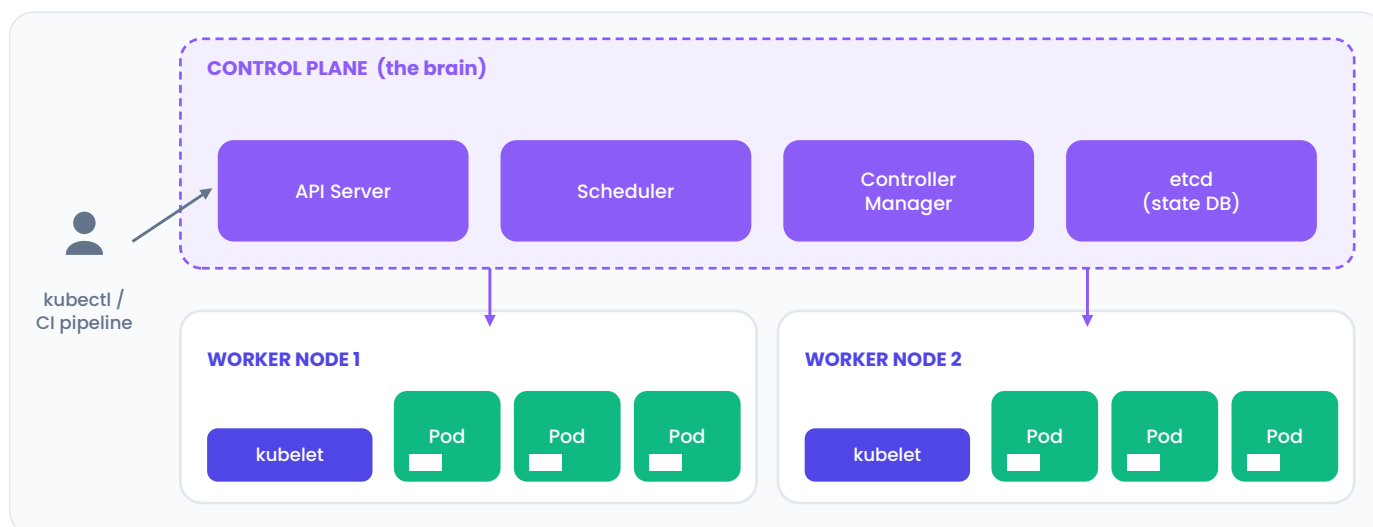


Fig 12. The control plane decides; worker nodes run your pods.

Object	Plain-English meaning
Pod	Smallest unit: one or more containers running together
Deployment	Keeps N copies of a pod running and handles rolling updates
Service	A stable address and load balancer for a group of pods
Ingress	Routes outside HTTP traffic to the right Service
ConfigMap / Secret	Configuration and sensitive values kept out of the image
HPA	Horizontal Pod Autoscaler: adds or removes pods by load

Five commands to start

```
kubectl apply -f deployment.yaml    # create or update
kubectl get pods                    # what is running?
kubectl logs <pod>                  # read output
kubectl scale deploy web --replicas=5
kubectl rollout undo deploy web     # roll back
```

09

CI/CD & Deployments

Ship small, ship safe

CI automatically builds and tests every change. **CD** automatically delivers it to users. Small, frequent, automated releases are **safer** than rare, big, manual ones.

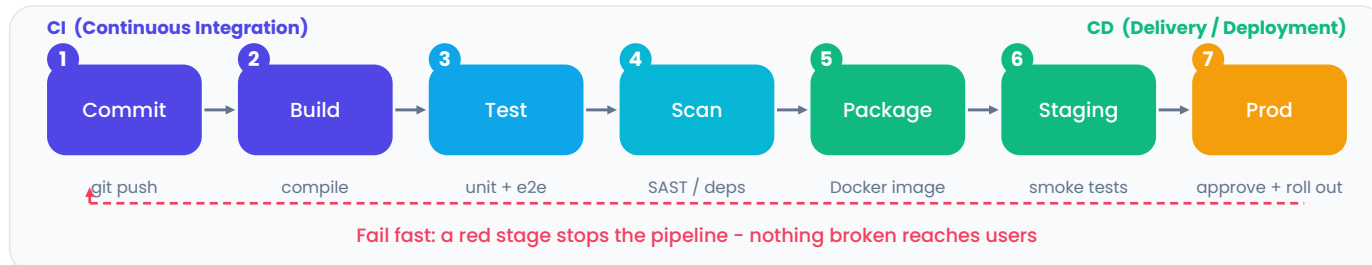


Fig 13. A typical pipeline. Every stage is a gate.

Pipeline rules of thumb

- **Build once, deploy many.** Promote the *same* image from staging to production.
- **Keep it fast.** Under ~10 minutes, or people stop trusting it.
- **Automate rollback.** Always know how to return to the last good version.
- **Tools:** GitHub Actions, GitLab CI, Jenkins, CircleCI, Argo CD.

Release strategies

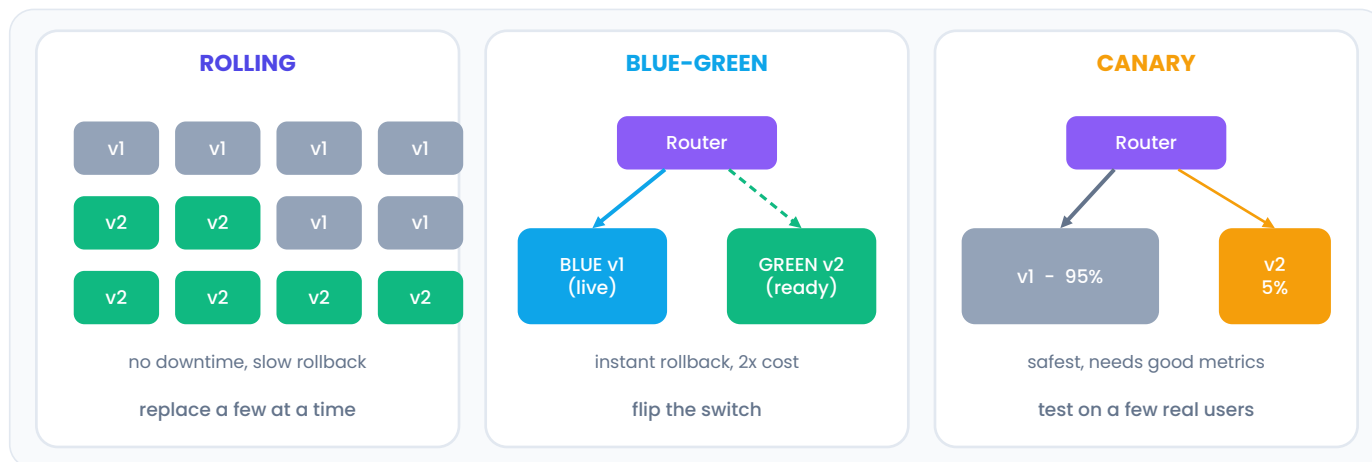


Fig 14. Three ways to release without a big bang.

Strategy	Risk	Rollback	Extra cost
Rolling	Medium	Slow (re-roll old version)	None
Blue-green	Low	Instant (flip back)	Double the environment
Canary	Lowest	Fast (stop sending traffic)	Needs strong monitoring

PRO TIP

Feature flags let you deploy code switched OFF, then turn it on for a few users. Deploying and releasing become two separate, safer steps.

10

IaC, GitOps & Environments

Infrastructure as reviewed code

Infrastructure as Code (IaC) means describing servers, networks and databases in files, stored in Git, reviewed and applied by tools. No more clicking in a console and hoping to remember what you did.

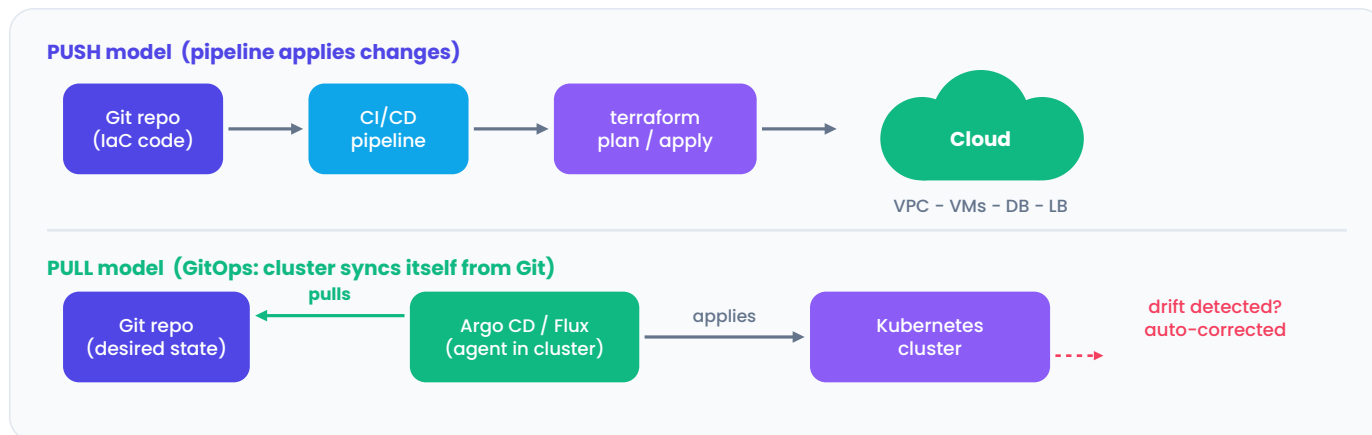


Fig 15. Push (pipeline applies) vs pull (GitOps agent syncs).

```
# Terraform: declare what you want
resource "aws_instance" "web" {
  ami          = "ami-0abc1234"
  instance_type = "t3.small"
  count        = 3
}
```

Why it wins

- **Repeatable:** build an identical environment in minutes.
- **Reviewable:** changes go through pull requests, with history and rollback.
- **Drift-proof:** tools detect and fix manual changes.
- **Idempotent:** running it twice gives the same result.

Tools by job

Job	Tools
Provision cloud resources	Terraform / OpenTofu, Pulumi, CloudFormation, Bicep
Configure machines	Ansible, Chef, Puppet
Package apps	Docker, Helm, Kustomize
GitOps for Kubernetes	Argo CD, Flux
Secrets	HashiCorp Vault, AWS Secrets Manager, Sealed Secrets

Environments

Promote changes through **dev** (experiments) then **staging** (production-like tests) then **production** (real users). Keep them as similar as possible and driven by the same code, only with different config values.

WATCH OUT

Never commit passwords, API keys or tokens to Git. Use a secrets manager and rotate keys regularly.

11

Observability & SRE

Logs, metrics, traces, SLOs

Observability is your ability to understand what is happening inside a system from the outside. If you cannot see it, you cannot fix it.

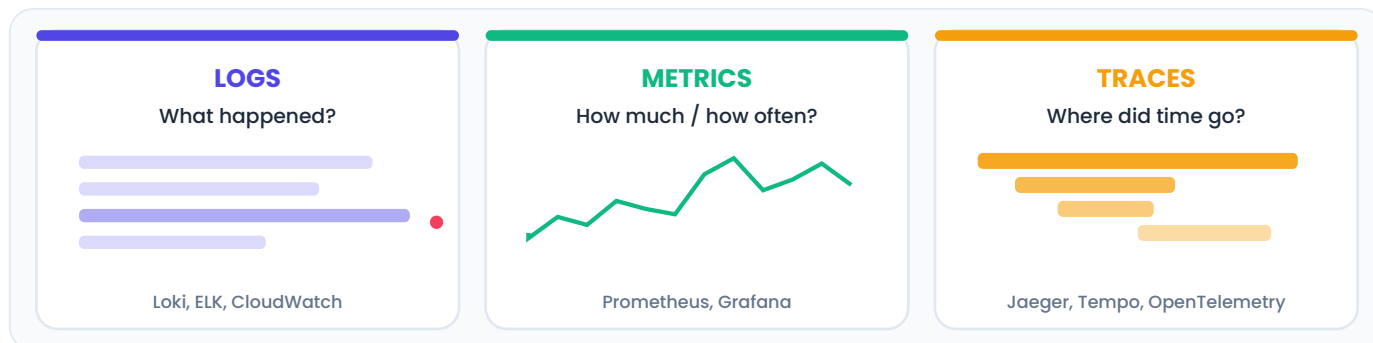


Fig 16. The three pillars of observability.

The four golden signals

Signal	Ask	Example
Latency	How long do requests take?	p95 = 240 ms
Traffic	How much demand is there?	1,200 requests / sec
Errors	What fraction fails?	0.2% HTTP 5xx
Saturation	How full is the system?	CPU 85%, disk 90%

SLI, SLO, SLA and the error budget

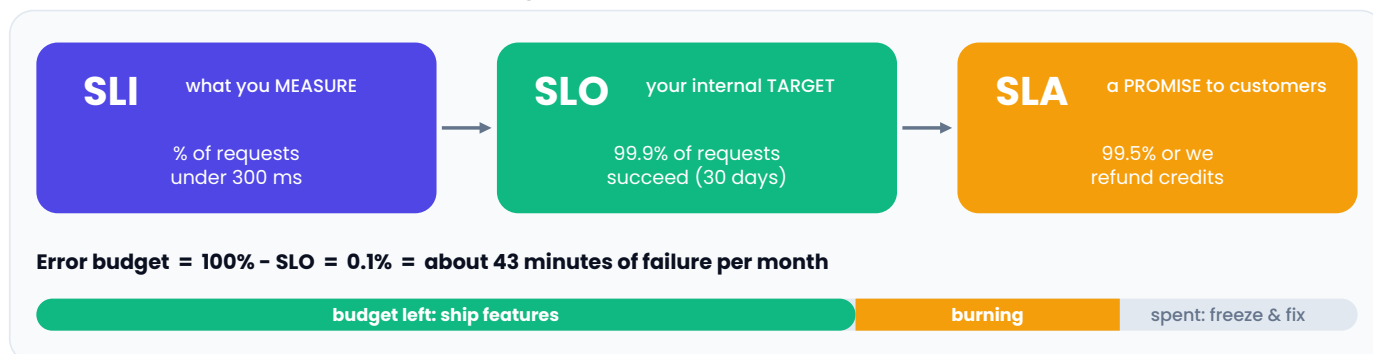


Fig 17. Measure it, target it, promise it. The error budget balances speed and stability.

PRO TIP

Alert on symptoms users feel (errors, slowness), not on every internal blip. Every alert should be urgent, real and actionable, or it is noise that causes alert fatigue. After an incident, write a **blameless post-mortem**.

12

Resilience & Recovery

Design for failure, plan for disaster

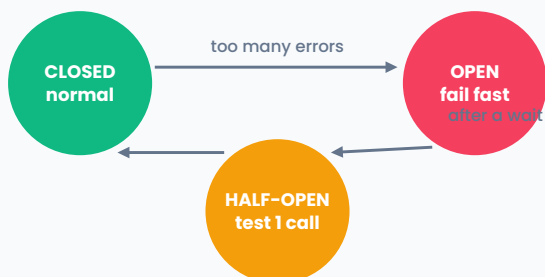


Things **will** fail: disks, networks, deployments, people. Good design makes failure small, contained and recoverable.

Resilience patterns

Pattern	What it does
Redundancy	Run 2+ of everything, across multiple Availability Zones
Health checks	Detect sick instances; restart or remove them automatically
Timeouts	Never wait forever for a slow dependency
Retries + backoff	Try again with growing delays and random jitter
Circuit breaker	Stop calling a failing service so it can recover
Rate limiting	Cap requests per user to protect the system
Bulkheads	Isolate parts so one failure cannot sink everything
Graceful degradation	Lose a feature, not the whole site (e.g. hide recommendations)
Idempotency	Repeating a request has the same effect as doing it once

CIRCUIT BREAKER



RETRY WITH BACKOFF + JITTER

Wait longer each time and add random jitter, so clients do not all retry at once.



Fig 18. A circuit breaker and exponential backoff.

Disaster recovery (DR)

A disaster can be a deleted database, a bad deploy or a whole region going down. Two numbers define your plan: **RPO** (how much data you can afford to lose) and **RTO** (how long you can afford to be down).

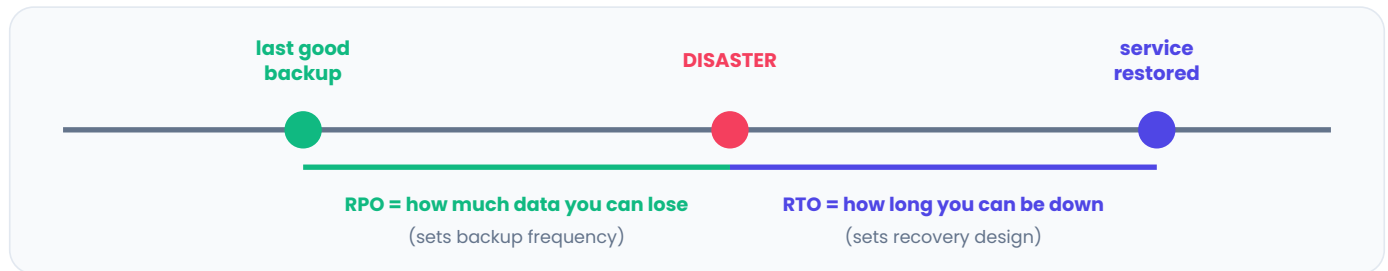


Fig 19. RPO and RTO define your recovery goals.

DR strategy	Recovery speed	Cost
Backup & restore	Hours	\$
Pilot light (tiny core always on)	Tens of minutes	\$\$
Warm standby (scaled-down copy)	Minutes	\$\$\$
Active-active (multi-region)	Near zero	\$\$\$\$

TRY IT

A backup you have never restored is only a hope. Schedule restore drills and **chaos tests** (deliberately break things in a safe way) to find weaknesses before users do.

When an incident happens

Step	What to do
1. Detect	An alert fires from monitoring (not a customer tweet)
2. Triage	How bad is it? Who is affected? Assign an incident lead
3. Mitigate	Stop the pain first: roll back, fail over, scale up. Fix the root cause later
4. Communicate	Update the status page and stakeholders regularly
5. Learn	Blameless post-mortem: what happened, why, and what we will change

13

Security & Cost Basics

Defense in depth, spend wisely

Security is not one wall. It is many layers, so if one fails the next still protects you.

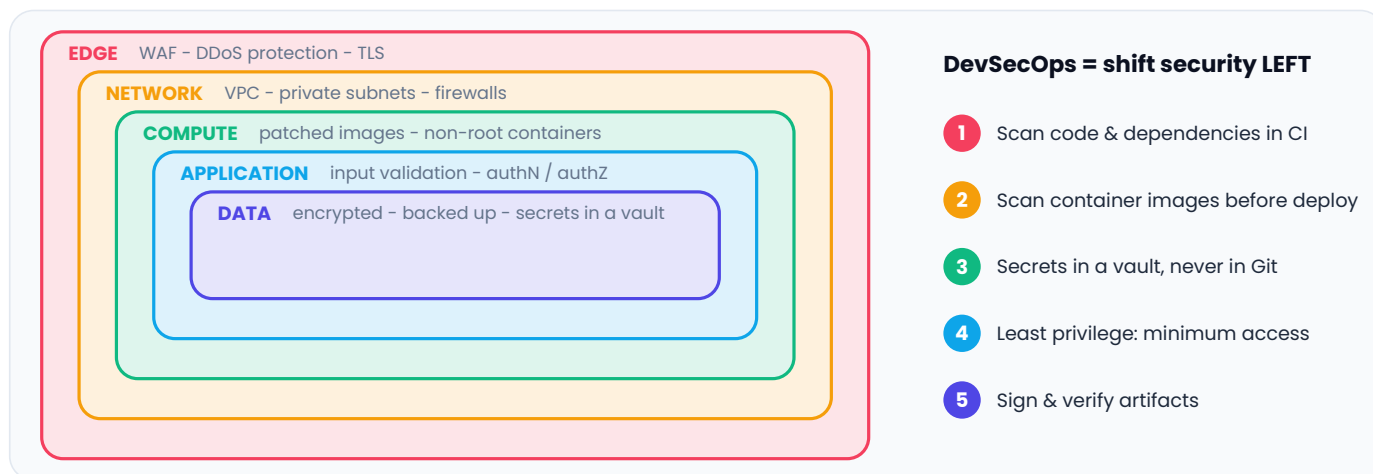


Fig 20. Defense in depth, plus shifting security checks earlier in the pipeline.

Security basics for every system

- **Least privilege:** give people and services only the access they need.
- **Encrypt everywhere:** TLS in transit, encryption at rest.
- **Patch and scan:** keep OS, images and dependencies updated.
- **Strong identity:** SSO, MFA, short-lived credentials instead of long-lived keys.
- **Audit logs:** know who did what, and when.

Cost basics (FinOps)

Lever	Idea
Right-size	Match instance size to real usage; most servers are over-provisioned
Auto-scale	Shrink at night and on weekends; pay for what you use
Pricing models	Reserved or savings plans for steady load; spot instances for fault-tolerant jobs
Storage tiers	Move old data to cheaper cold storage with lifecycle rules
Visibility	Tag resources, set budgets and alerts, review bills monthly

KEY IDEA

Reliability, security and cost pull against each other. Write the trade-off down so the whole team understands *why* a choice was made.

14

Putting It All Together

A reference architecture and a method

Here every earlier idea appears in one picture: a typical, healthy web application.

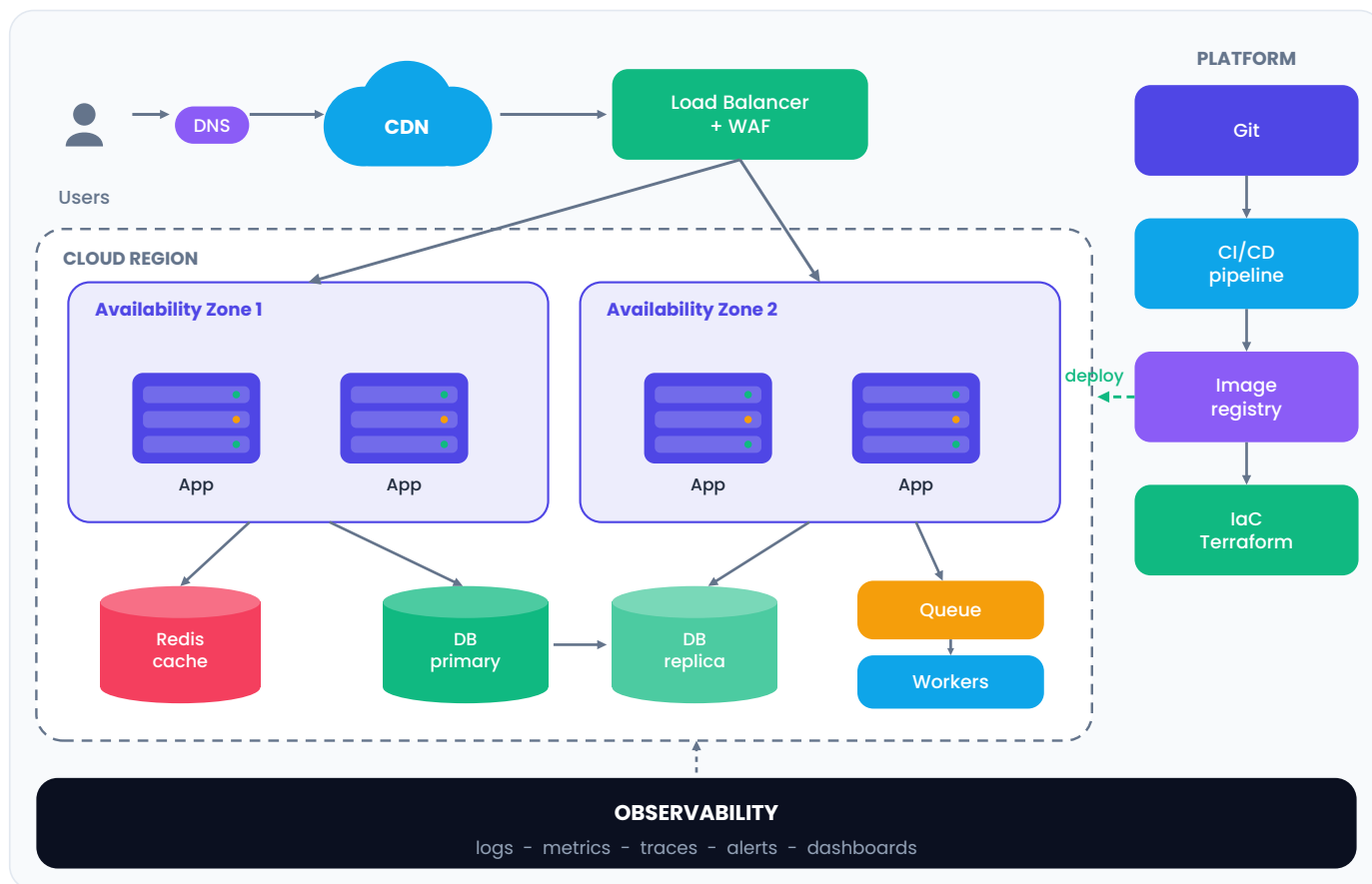


Fig 21. A reference architecture: redundant, observable, and delivered by an automated pipeline.

A simple method for any design problem



Fig 22. Five steps to approach any design question.

Back-of-envelope example

Step	Calculation	Result
Daily load	1 M users x 20 requests each	20 M requests / day
Average rate	20 M / 86,400 seconds	~230 requests / sec
Peak rate	Assume peak = 3x average	~700 requests / sec
Servers	One server handles ~250 req/s (assumed)	3 needed, plus 1 spare (N+1)
Placement	Survive losing a zone	Spread across 2+ Availability Zones

15

Cheat Sheet & Roadmap

Revise fast, learn next



Problem to solution

If you see...	Reach for...
Slow reads	Cache, CDN, read replicas, indexes
Traffic spikes	Auto-scaling, queues, rate limiting
Single point of failure	Redundancy, multi-AZ, load balancer pairs
Risky releases	CI/CD, canary, blue-green, feature flags
Cannot debug production	Centralised logs, metrics, tracing
Environments differ	IaC, containers, GitOps
One failure spreads	Timeouts, circuit breakers, bulkheads
Fear of data loss	Backups, replication, restore drills, PITR
Leaked secrets	Vault, short-lived credentials, secret scanning

Mini glossary

Term	Meaning	Term	Meaning
Latency	Time for one request	Throughput	Requests handled per second
SPOF	Single point of failure	Idempotent	Safe to repeat
Failover	Auto-switch to a backup	Drift	Real state differs from code
AZ	Availability Zone: isolated data centre	MTTR	Mean time to recover
Pod	Smallest Kubernetes unit	TTL	Time before data expires

Your learning roadmap

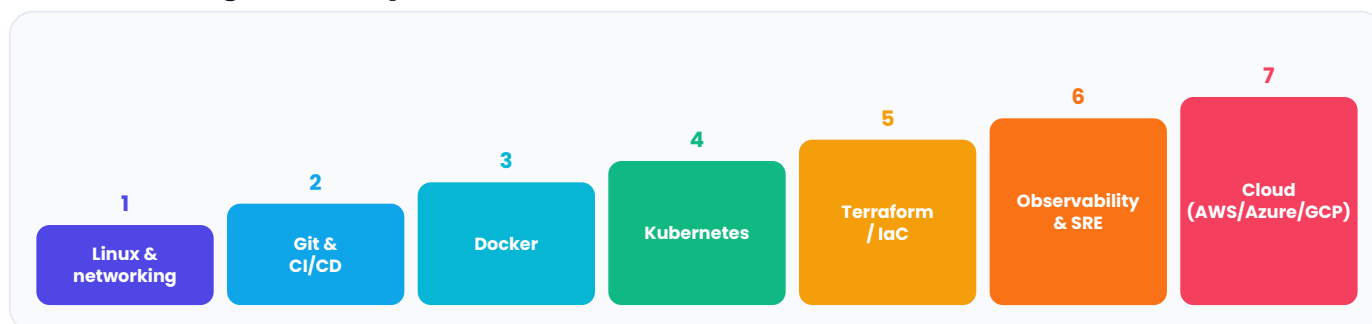


Fig 23. Learn in this order; practise each with a small project.

TRY IT

Mini project: containerise a small web app, build a CI/CD pipeline for it, deploy 3 replicas behind a load balancer with Terraform, add Prometheus + Grafana, then kill a server and watch it recover.